

// SMART MANUFACTURING AND PRODUCTION AUTOMATION SYSTEM

```
#include<iostream>

#include<string>

#include<thread> // pause the code for specific time (used in wrong password
attempts)

#include<cctype>

#include<chrono> // time duration like seconds and min, hr.

using namespace std;

void menu();

// 1. FACTORY MANAGEMENT CLASS

class factory
{
    private:
    string factory_employee_id[100];
    string factory_employee_name[100];
    int factory_age[100];
    string factory_gender[100];
    string factory_phone_no[100];
    string factory_address[100];
    string factory_department[100];
    string factory_designation[100];
    string factory_joining_date[100];
    string factory_shift[100];
    string factory_employee_status[100];
    double factory_basic_salary[100] = {};
    double factory_allowance[100] = {};
    double factory_deduction[100] = {};
    double factory_net_salary[100] = {};
    int factory_working_days[100] = {};
```

```

int factory_present_days[100] = {};
int factory_absent_days[100] = {};
int factory_employee_count = 0;
int i;

public:
    string factory_name;

    // USE OF CONSTRUCTURE
    factory() : total_employees(0), total_production(0) {}
    factory(string factory_name): factory_name(factory_name), total_employees(0),
    total_production(0)
    {
    }

private: //use of private
    string location;
    string owner_name;
    int total_production;
    int total_employees;
    string phone_no;

public:
    void register_factory()
    {
        cout<<"=====COMPANY REGISTRATION=====\\n\\n";
        cout<<"enter factory name\\n";
        getline(cin>>ws, factory_name); // ws = white space
        cout<<"enter owner name\\n";
        getline(cin>>ws, owner_name);
    }

```

wrong:

```
    cout<<"enter phone number\n";
    getline(cin>>ws, phone_no);

    if(phone_no.length() != 14) // no of phone number will be 14 while counting the white
space
    {
        cout<<"phone number format will be (+xx xxxxxxxxxx)\n";
        goto wrong;// use of goto statement
    }
    else
    {
        goto correct;
    }
```

correct:

```
    cout<<"enter company location (state)\n";
    cin>>location;

    cout<<"enter total number of employees in your company (maximum)\n";
    cin>>total_employees;

    cout<<"enter total types of machine your company manufacture\n";
    cin>>total_production;

}
```

public:

```
void add_factory_employee()
{
    int number;

    cout<<"\nenter number of employees to add during registration: ";
    cin>>number;
```

```

if(number <= 0 || number > total_employees)
{
    cout<<"\ninvalid number of employees!";
    return;
}
for(int i = 0; i < number; i++) // use of for loop
{
    cout << "\n===== \n";
    cout << "    employee " << factory_employee_count + 1 << " \n";
    cout << "===== \n";
    cout << "\nenter employee id: ";
    cin >> factory_employee_id[factory_employee_count]; // factory_employee_count
showing the position
    cout << "enter employee name: ";
    getline(cin>>ws,factory_employee_name[factory_employee_count]);
    cout << "enter age: ";
    cin >> factory_age[factory_employee_count];
    cout << "enter gender: ";
    cin >> factory_gender[factory_employee_count];
    cout << "enter phone number: ";
    getline(cin>>ws,factory_phone_no[factory_employee_count]);
    cout << "enter address: ";
    getline(cin>>ws,factory_address[factory_employee_count]);
    cout << "enter department: ";
    getline(cin>>ws,factory_department[factory_employee_count]);
    cout << "enter designation: ";
    getline(cin>>ws,factory_designation[factory_employee_count]);
    cout << "enter joining date: ";

```

```

        getline(cin>>ws,factory_joining_date[factory_employee_count]);
        cout << "enter shift: ";
        cin >> factory_shift[factory_employee_count];
        cout << "enter employee status: ";
        getline(cin>>ws,factory_employee_status[factory_employee_count]);
        factory_employee_count++; // increase the value by 1
        cout << "\nemployee added successfully!";
    }

    cout << "\n\n total employees added: " << factory_employee_count;
}

public:
    void show_factory()
    {
        cout<<"=====
=====\\n\\n";

        cout<<"          COMPANY NAME = "<<factory_name<<endl;
        cout<<"          OWNER NAME = "<<owner_name<<endl;
        cout<<"          PHONE NUMBER = "<<phone_no<<endl;
        cout<<"          COMPANY LOCATION = "<<location<<endl;
        cout<<"          NUMBER OF EMPLOYEES = "<<total_employees<<endl;
        cout<<"          NUMBER OF MACHINE MANUFACTURE BY COMPANY =
"<<total_production<<endl;

        cout<<"\\n\\n=====
=====\\n";

    }

public:
    void show_factory_employee()
    {

```

```

if(factory_employee_count == 0)
{
    cout << "\nno employee added!";
    return;
}
for(int i = 0; i < factory_employee_count; i++)
{
    cout << "\n===== \n";
    cout << "    employee " << i + 1 << "\n";
    cout << "===== \n";
    cout << "\nemployee id    : " << factory_employee_id[i];
    cout << "\nemployee name  : " << factory_employee_name[i];
    cout << "\nage          : " << factory_age[i];
    cout << "\ngender        : " << factory_gender[i];
    cout << "\nphone number   : " << factory_phone_no[i];
    cout << "\naddress       : " << factory_address[i];
    cout << "\n\ndepartment    : " << factory_department[i];
    cout << "\ndesignation    : " << factory_designation[i];
    cout << "\njoining date   : " << factory_joining_date[i];
    cout << "\nshift         : " << factory_shift[i];
    cout << "\nemployee status : " << factory_employee_status[i];
    cout << "\n===== \n";
}
}

```

```

int get_factory_employee_count()
{
    return factory_employee_count;
}

```

```
}
```

```
public:
```

```
void add_one_employee_from_management()
```

```
{
```

```
    if(factory_employee_count >= 100)
```

```
    {
```

```
        cout << "\nmaximum storage capacity of 100 employees reached!";
```

```
        return;
```

```
    }
```

```
    cout << "\n===== \n";
```

```
    cout << "    employee " << factory_employee_count + 1 << "\n";
```

```
    cout << "===== \n";
```

```
    cout << "\nenter employee id: ";
```

```
    cin >> factory_employee_id[factory_employee_count];
```

```
    cout << "enter employee name: ";
```

```
    getline(cin >> ws, factory_employee_name[factory_employee_count]);
```

```
    cout << "enter age: ";
```

```
    cin >> factory_age[factory_employee_count];
```

```
    cout << "enter gender: ";
```

```
    cin >> factory_gender[factory_employee_count];
```

```
    cout << "enter phone number: ";
```

```
    getline(cin >> ws, factory_phone_no[factory_employee_count]);
```

```
    cout << "enter address: ";
```

```
    getline(cin >> ws, factory_address[factory_employee_count]);
```

```
    cout << "enter department: ";
```

```
    getline(cin >> ws, factory_department[factory_employee_count]);
```

```

cout << "enter designation: ";
getline(cin >> ws, factory_designation[factory_employee_count]);
cout << "enter joining date: ";
getline(cin >> ws, factory_joining_date[factory_employee_count]);
cout << "enter shift: ";
cin >> factory_shift[factory_employee_count];
cout << "enter employee status: ";
getline(cin >> ws, factory_employee_status[factory_employee_count]);
factory_employee_count++;
if(factory_employee_count > total_employees)
{
    total_employees = factory_employee_count;
}
cout << "\nemployee added successfully!";
}

public:
void employee_management_add()
{
    int number;
    if(factory_employee_count >= total_employees)
    {
        cout << "\nmaximum number of employees reached!";
        return;
    }
    cout << "\nenter number of employees you want to add: ";
    cin >> number;
    if(number <= 0)
    {

```

```

    cout << "\ninvalid number of employees!";
    return;
}
if(factory_employee_count + number > total_employees)
{
    cout << "\nyou can only add "
        << total_employees - factory_employee_count
        << " more employee(s)!";
    return;
}
for(int i = 0; i < number; i++)
{
    cout << "\n===== \n";
    cout << "    employee " << factory_employee_count + 1 << "\n";
    cout << "===== \n";
    cout << "\nenter employee id: ";
    cin >> factory_employee_id[factory_employee_count];
    cout << "enter employee name: ";
    getline(cin>>ws,factory_employee_name[factory_employee_count]);
    cout << "enter age: ";
    cin >> factory_age[factory_employee_count];
    cout << "enter gender: ";
    cin >> factory_gender[factory_employee_count];
    cout << "enter phone number: ";
    getline(cin>>ws,factory_phone_no[factory_employee_count]);
    cout << "enter address: ";
    getline(cin>>ws,factory_address[factory_employee_count]);
    cout << "enter department: ";

```

```

        getline(cin>>ws,factory_department[factory_employee_count]);
        cout << "enter designation: ";
        getline(cin>>ws,factory_designation[factory_employee_count]);
        cout << "enter joining date: ";
        getline(cin>>ws,factory_joining_date[factory_employee_count]);
        cout << "enter shift: ";
        cin >> factory_shift[factory_employee_count];
        cout << "enter employee status: ";
        getline(cin>>ws,factory_employee_status[factory_employee_count]);
        factory_employee_count++;
        cout << "\nemployee added successfully!";
    }

    cout << "\n\n total employees added: " << factory_employee_count;
}

public:
void calculate_factory_salary()
{
    if(factory_employee_count == 0)
    {
        cout << "\nno employee added!";
        return;
    }

    string id;
    int index = -1;
    cout << "\n enter employee id: ";
    cin >> id;
    for(int i = 0; i < factory_employee_count; i++)
    {

```

```
    if(factory_employee_id[i] == id)
    {
        index = i;
        break;
    }
}
if(index == -1)
{
    cout << "\nemployee not found!";
    return;
}
cout << "\nenter basic salary: ";
cin >> factory_basic_salary[index];

if(factory_basic_salary[index] < 0)
{
    cout << "\ninvalid basic salary!";
    return;
}
cout << "enter allowance: ";
cin >> factory_allowance[index];
if(factory_allowance[index] < 0)
{
    cout << "\ninvalid allowance!";
    return;
}
cout << "enter deduction: ";
cin >> factory_deduction[index];
```

```

    if(factory_deduction[index] < 0)
    {
        cout << "\ninvalid deduction!";
        return;
    }
    factory_net_salary[index] =
        factory_basic_salary[index] +
        factory_allowance[index] -
        factory_deduction[index];
    cout << "\nnet salary = "
        << factory_net_salary[index] << endl;
}

public:
void factory_attendance()
{
    if(factory_employee_count == 0)
    {
        cout << "\nno employee added!";
        return;
    }
    string id;
    int index = -1;
    cout << "\nenter employee id: ";
    cin >> id;
    for(int i = 0; i < factory_employee_count; i++)
    {
        if(factory_employee_id[i] == id)
        {

```

```

        index = i; // copying the value of i in the index
        break;
    }
}
if(index == -1)
{
    cout << "\nemployee not found!";
    return;
}
cout << "\nenter working days: ";
cin >> factory_working_days[index];
if(factory_working_days[index] <= 0)
{
    cout << "\ninvalid working days!";
    return;
}
cout << "enter present days: ";
cin >> factory_present_days[index];
if(factory_present_days[index] < 0 ||
    factory_present_days[index] > factory_working_days[index])
{
    cout << "\npresent days cannot be greater than working days!";
    return;
}
factory_absent_days[index] =
    factory_working_days[index] -
    factory_present_days[index];
cout << "\nattendance updated successfully!";

```

```

    cout << "\nworking days : " << factory_working_days[index];
    cout << "\npresent days : " << factory_present_days[index];
    cout << "\nabsent days : " << factory_absent_days[index];
}

public:
void show_factory_employee_management()
{
    if(factory_employee_count == 0)
    {
        cout << "\nno employee added!";
        return;
    }
    for(int i = 0; i < factory_employee_count; i++)
    {
        cout << "\n===== \n";
        cout << "    employee " << i + 1 << "\n";
        cout << "===== \n";
        cout << "\nemployee id    : " << factory_employee_id[i];
        cout << "\nemployee name  : " << factory_employee_name[i];
        cout << "\nage          : " << factory_age[i];
        cout << "\ngender        : " << factory_gender[i];
        cout << "\nphone number  : " << factory_phone_no[i];
        cout << "\naddress       : " << factory_address[i];
        cout << "\n\ndepartment   : " << factory_department[i];
        cout << "\ndesignation   : " << factory_designation[i];
        cout << "\njoining date  : " << factory_joining_date[i];
        cout << "\nshift        : " << factory_shift[i];
        cout << "\nemployee status : " << factory_employee_status[i];
    }
}

```

```

        cout << "\n\n----- salary details ----- ";
        cout << "\nbasic salary   : " << factory_basic_salary[i];
        cout << "\nallowance     : " << factory_allowance[i];
        cout << "\ndeduction      : " << factory_deduction[i];
        cout << "\nnet salary      : " << factory_net_salary[i];
        cout << "\n\n----- attendance details --- ";
        cout << "\nworking days : " << factory_working_days[i];
        cout << "\npresent days : " << factory_present_days[i];
        cout << "\nabsent days   : " << factory_absent_days[i];
        cout << "\n=====\\n";
    }
}

public:
void factory_employee_report()
{
    if(factory_employee_count == 0)
    {
        cout << "\nno employee added!";
        return;
    }

    for(int i = 0; i < factory_employee_count; i++)
    {
        cout << "\n=====\\n";
        cout << "      employee report " << i + 1 << "\\n";
        cout << "=====\\n";
        cout << "\nemployee id   : " << factory_employee_id[i];
        cout << "\nemployee name   : " << factory_employee_name[i];
    }
}

```

```

cout << "\ndepartment    :" << factory_department[i];
cout << "\ndesignation    :" << factory_designation[i];
cout << "\nemployee status  :" << factory_employee_status[i];
cout << "\n\n----- attendance report ---";
cout << "\nworking days   :" << factory_working_days[i];
cout << "\npresent days    :" << factory_present_days[i];
cout << "\nabsent days     :" << factory_absent_days[i];
if(factory_working_days[i] > 0)
{
    float attendance_percentage;
    attendance_percentage =
        ((float)factory_present_days[i] /
         factory_working_days[i]) * 100;
    cout << "\nattendance      : "
        << attendance_percentage << "%";
    if(attendance_percentage >= 90)
    {
        cout << "\nattendance status : excellent";
    }
    else if(attendance_percentage >= 75)
    {
        cout << "\nattendance status : good";
    }
    else
    {
        cout << "\nattendance status : low";
    }
}
}

```

```

else
{
    cout << "\nattendance      : not available";
}

cout << "\n\n----- salary report----- ";
cout << "\nbasic salary   : " << factory_basic_salary[i];
cout << "\nallowance     : " << factory_allowance[i];
cout << "\ndeduction       : " << factory_deduction[i];
cout << "\nnet salary      : " << factory_net_salary[i];
cout << "\n=====\\n";
}
}

string get_phone_no()
{
    return phone_no;
}

int get_total_employees()
{
    return total_employees;
}

void set_total_employees(int total)
{
    total_employees = total;
}

};

// 2. MACHINE MANAGEMENT CLASS
class machine_management : public factory
{

```

private:

```
string machine_id;  
string machine_name;  
string machine_type;  
string manufacturer;  
float power_consumption = 0;  
float temperature = 0;  
float speed = 0;  
float vibration = 0;  
float machine_expense = 0;  
float operating_hr = 0;  
string status;
```

public:

```
machine_management() : factory() {}  
machine_management(string factory_name) : factory(factory_name) {}
```

public:

```
void machine()  
{  
cout<<"=====\\n";  
  
    cout<<"=====MACHINE MANAGEMENT SYSTEM=====\\n";  
    cout<<"enter machine id\\n";  
    getline(cin>>ws, machine_id);  
    cout<<"enter machine name\\n";  
    getline(cin>>ws, machine_name);  
    cout<<"enter machine type\\n";  
    getline(cin>>ws, machine_type);  
    cout<<"enter total manufacturing expense\\n";  
    cin>>machine_expense;
```

no:

```
cout<<"enter manufacturer (company name)\n";
getline(cin>>ws, manufacturer);// WS = WHITE SPACE
if(factory_name == manufacturer)
{
    goto yes;
}
else
{
    cout<<"wrong company name\n";
    goto no;
}
```

yes:

```
cout<<"enter power consumption\n";
cin>>power_consumption;
cout<<"enter temperature\n";
cin>>temperature;
cout<<"enter speed\n";
cin>>speed;
cout<<"enter vibration\n";
cin>>vibration;
cout<<"enter operating hour\n";
cin>>operating_hr;
}
```

public:

```
string get_machine_id()
{
    return machine_id;
}
```

```

    }

public:

    void showmachine()

    {
cout<<"=====
=====\\n\\n";

        cout<<"          MACHINE ID = "<<machine_id<<endl;
        cout<<"          MACHINE NAME = "<<machine_name<<endl;
        cout<<"          MACHINE TYPE = "<<machine_type<<endl;
        cout<<"          TOTAL MANUFACTURING EXPENSE =
"<<machine_expense<<endl;

        cout<<"          MACHINE MANUFACTURER = "<<manufacturer<<endl;
        cout<<"          POWER CONSUMPTION = "<<power_consumption<<"
KW"<<endl;

        cout<<"          MACHINE TEMPERATURE = "<<temperature<<" degree
celcius"<<endl;

        cout<<"          MACHINE SPEED = "<<speed<<" RPM"<<endl;
        cout<<"          MACHINE VIBRATION = "<<vibration<<" mm/s"<<endl;
        cout<<"          MACHINE OPERATING HOURS = "<<operating_hr<<"
HOURS"<<endl;

        // CRITICAL CHECK

        if(power_consumption > 25 || temperature > 85 ||speed > 2200 ||vibration > 7
||operating_hr > 1500)

        {
            status = "CRITICAL";

            cout<<"          MACHINE STATUS = "<<status<<endl;
            cout<<"          EMERGENCY SHUTDOWN STARTED\\n";
            cout<<"          MACHINE IS STOPPED\\n";
            cout<<"          MAINTENANCE IS REQUIRED\\n";

        }

```



```

class production_management : public factory
{
private:
    string production_id;
    string product_name;
    string product_type;
    int target_quantity = 0;
    int produced_quantity = 0;
    int rejected_quantity = 0;
    float production_hours = 0;
public:
    production_management() : factory() {}
    production_management(string factory_name) : factory(factory_name) {}
    void production()
    {

cout<<"=====
=====\\n";

    cout<<"===== PRODUCTION MANAGEMENT SYSTEM =====\\n";
    cout<<"enter production id\\n";
    getline(cin>>ws, production_id);
    cout<<"enter product name\\n";
    getline(cin>>ws, product_name);
    cout<<"enter product type\\n";
    getline(cin>>ws, product_type);
    cout<<"enter target quantity\\n";
    cin>>target_quantity;
    if(target_quantity <= 0)
    {

```

```

        cout<<"invalid target quantity\n";
        return;
    }
    cout<<"enter produced quantity\n";
    cin>>produced_quantity;
    if(produced_quantity < 0)
    {
        cout<<"invalid produced quantity\n";
        return;
    }
    rejected:
    cout<<"enter rejected quantity\n";
    cin>>rejected_quantity;
    if(rejected_quantity < 0 || rejected_quantity > produced_quantity)
    {
        cout<<"rejected quantity cannot be greater than produced quantity\n";
        goto rejected;
    }
    cout<<"enter production hours\n";
    cin>>production_hours;
    if(production_hours < 0)
    {
        cout<<"invalid production hours\n";
        return;
    }
}

void showproduction()
{

```

```

cout<<"=====
=====\\n\\n";

    cout<<"        PRODUCTION ID = "<<production_id<<endl;
    cout<<"        PRODUCT NAME = "<<product_name<<endl;
    cout<<"        PRODUCT TYPE = "<<product_type<<endl;
    cout<<"        TARGET QUANTITY = "<<target_quantity<<endl;
    cout<<"        PRODUCED QUANTITY = "<<produced_quantity<<endl;
    cout<<"        REJECTED QUANTITY = "<<rejected_quantity<<endl;
    cout<<"        PRODUCTION HOURS = "<<production_hours<<endl;
    int good_production = produced_quantity - rejected_quantity;
    cout<<"        GOOD PRODUCTION = "<<good_production<<endl;
    if(target_quantity > 0)
    {
        float efficiency =
            ((float)produced_quantity / target_quantity) * 100;

        cout<<"        PRODUCTION EFFICIENCY = "
            <<efficiency<<"%"<<endl;
    }

    cout<<"\\n\\n=====
=====\\n\\n";

}

};

// 4. LOGIN AND PASSWORD CLASS
class login
{
private:

```

```
string password;
string confirmpassword;
string created_password;
public:
    bool validate_password(string p)
    {
        bool has_upper = false;
        bool has_lower = false;
        bool has_digit = false;
        bool has_symbol = false;

        if(p.length() < 6)
        {
            cout<<"password must contain at least 6 characters\n";
            return false;
        }

        for(int i = 0; i < p.length(); i++)
        {
            if(isupper(p[i]))
            {
                has_upper = true;
            }

            else if(islower(p[i]))
            {
                has_lower = true;
            }
        }
    }
}
```

```
    else if(isdigit(p[i]))
    {
        has_digit = true;
    }
    else
    {
        has_symbol = true;
    }
}
```

```
if(has_lower == false)
{
    cout<<"password must contain at least one lower case letter\n";
    return false;
}
```

```
if(has_digit == false)
{
    cout<<"password must contain at least one digit\n";
    return false;
}
```

```
if(has_upper == false)
{
    cout<<"password must contain at least one upper case letter\n";
    return false;
}
```

```

    if(has_symbol == false)
    {
        cout<<"password must contain at least one symbol\n";
        return false;
    }

    return true;
}

public:
    void create()
    {
        cout<<"=====\\n";
        cout<<"=====LET'S CREATE THE PASSWORD FIRST=====\\n";
        cout<<"password must contain:\\n";
        cout<<"minimum 6 characters\\n";
        cout<<"at least one upper letter\\n";
        cout<<"at least one letter\\n";
        cout<<"at least one digit\\n";
        cout<<"at least one symbol\\n";
        cout<<"=====\\n";

        again:
        cout<<"enter new password\\n";
        cin>>password;
        if(validate_password(password) == false)
        {
            cout<<"invalid password format\\n";
            cout<<"please create password again\\n";
            goto again;
        }
    }

```

```

    }

    cout<<"confirm password\n";
    cin>>confirmpassword;

    if(password == confirmpassword)
    {
        created_password = password;
        cout<<"password created successfully\n";
    }
    else
    {
        cout<<"password not matched\n";
        cout<<"please create password again\n";
        goto again;
    }
}

bool login_id(string correct_phone)
{
    int attempt = 0;
    string entered_password;
    int option;
    login_again:
    attempt = 0;
    cout<<"=====\\n";

    cout<<"          LOGIN\\n";

    cout<<"=====\\n";

    wrong_password:
    cout<<"enter password\\n";

```

```

cin>>entered_password;
if(entered_password == created_password)
{
    cout<<"login successful\n";
    cout<<"welcome to smart manufacturing system\n";
    return true;
}
else
{
    attempt++;
    cout<<"wrong password\n";
    if(attempt < 3)
    {
        cout<<"remaining attempts = "<<3-attempt<<endl;
        goto wrong_password;
    }
    else
    {
        cout<<"=====\\n";
        cout<<"you have entered wrong password 3 times\\n";
        cout<<"          1. change password\\n";
        cout<<"          2. wait / lock system\\n";
        cout<<"enter your choice\\n";
        cin>>option;
        if(option == 1)
        {
            change_password(correct_phone);
            cout<<"please login again with your new password\\n";

```

```

        goto login_again;
    }
    else if(option == 2)
    {
        cout<<"system is locked for 10 seconds\n";
        for(int i=10; i>=1; i--)
        {
            cout<<"please wait "<<i<<" seconds\r";
            this_thread::sleep_for(chrono::seconds(1)); // wait for 10 sec
        }
        cout<<"\nsystem unlocked\n";
        goto login_again;
    }
    else
    {
        cout<<"invalid input\n";
        goto wrong_password;
    }
}

}

}

void change_password(string correct_phone)
{
    string entered_phone;
    cout<<"===== \n";
    cout<<"          CHANGE PASSWORD\n";

    cout<<"===== \n";

```

```
cout<<"enter factory phone number\n";
getline(cin>>ws, entered_phone);
if(entered_phone == correct_phone)
{
    cout<<"phone number verified\n";
again_password:
    cout<<"enter new password\n";
    cin>>password;
    if(validate_password(password) == false)
    {
        cout<<"invalid password format\n";
        cout<<"please enter password again\n";
        goto again_password;
    }
    cout<<"confirm new password\n";
    cin>>confirmpassword;
    if(password == confirmpassword)
    {
        created_password = password;
        cout<<"password changed successfully\n";
    }
    else
    {
        cout<<"password not matched\n";
        cout<<"please enter password again\n";
        goto again_password;
    }
}
```

```

    else
    {
        cout<<"wrong phone number\n";
        cout<<"password cannot be changed\n";
    }
}

};

// 5. INVENTORY CLASS

class inventory : public factory
{
private:
    // use of array
    string material_id[100];
    string material_name[100];
    float quantity[100];
    string unit[100];
    float minimum_stock[100];
    float unit_price[100];
    string supplier_name[100];
    string stock_status[100];
    float material_quantity = 0;
    int product_number = 0;
    int i;
public:
    inventory() : factory() {}
    inventory(string factory_name) : factory(factory_name) {}
public:
    void add_stock()

```

```

{

cout<<"=====
=====\\n";

    cout<<"===== INVENTORY MANAGEMENT SYSTEM =====\\n";
    cout<<"enter number of products/material you want to enter\\n";
    cin>>product_number;
    if(product_number <= 0 || product_number > 100)
    {
        cout<<"invalid number of materials\\n";
        product_number = 0;
        return;
    }
    for(i=0; i<product_number; i++)
    {
        cout<<"\\n===== MATERIAL "<i+1<<" =====\\n";
        cout<<"enter material id\\n";
        cin>>material_id[i];
        cout<<"enter material name\\n";
        getline(cin>>ws, material_name[i]);
        cout<<"enter quantity to add\\n";
        cin>>quantity[i];
        if(quantity[i] < 0)
        {
            cout<<"invalid quantity\\n";
            quantity[i] = 0;
        }
        cout<<"enter unit\\n";
        cin>>unit[i];
    }
}

```

```
cout<<"enter minimum stock range\n";
cin>>minimum_stock[i];
cout<<"enter unit price\n";
cin>>unit_price[i];
noo:

cout<<"enter supplier name\n";
getline(cin>>ws, supplier_name[i]);

if(factory_name == supplier_name[i])
{
    cout<<"supplier name noted\n";
}
else
{
    cout<<"wrong company name\n";
    goto noo;
}
if(quantity[i] < minimum_stock[i])
{
    stock_status[i] = "LOW STOCK";
}
else if(quantity[i] == minimum_stock[i])
{
    stock_status[i] = "MINIMUM STOCK";
}

else
{

```

```

        stock_status[i] = "STOCK FULL";
    }
}

cout<<"\n all materials added successfully\n";
}

public:

bool sell_material(string sell_id, string sell_name, int sell_quantity, float
Cselling_price)
{
    if(product_number == 0)
    {
        cout << "\nno materials available in inventory!\n";
        return false;
    }

    for(int i = 0; i < product_number; i++)
    {
        // CHECK MATERIAL ID
        if(sell_id == material_id[i])
        {
            // CHECK MATERIAL NAME
            if(sell_name != material_name[i])
            {
                cout << "\nwrong material name!";
                cout << "\nmaterial ID exists, but material name does not match.\n";
                return false;
            }

            // CHECK QUANTITY

```

```
if(sell_quantity <= 0)
{
    cout << "\ninvalid selling quantity!\n";
    return false;
}

if(sell_quantity > quantity[i])
{
    cout << "\ninsufficient stock!";
    cout << "\navailable stock = " << quantity[i] << endl;
    cout << "requested quantity = " << sell_quantity << endl;
    return false;
}

// GET PRICE FROM INVENTORY
selling_price = unit_price[i];

// REDUCE STOCK
quantity[i] = quantity[i] - sell_quantity;

// UPDATE STOCK STATUS
if(quantity[i] < minimum_stock[i])
{
    stock_status[i] = "LOW STOCK";
}
else if(quantity[i] == minimum_stock[i])
{
    stock_status[i] = "MINIMUM STOCK";
}
```

```

        else
        {
            stock_status[i] = "STOCK FULL";
        }

        cout << "\nproduct verified successfully!";
        cout << "\nstock available for sale.";
        cout << "\nremaining stock = " << quantity[i] << endl;

        return true;
    }
}

// ID NOT FOUND
cout << "\nmaterial ID not found in inventory!\n";
return false;
}

public:
    void showentry()
    {

        cout<<"=====
=====\\n";

        cout<<"          ALL MATERIALS\\n";

        cout<<"=====
=====\\n\\n";

        if(product_number == 0)

```

```

{
    cout<<"no materials available\n";
    return;
}

for(i=0; i<product_number; i++)
{
    cout<<"===== MATERIAL "<<i+1<<" =====\n\n";
    cout<<"        material id = "<<material_id[i]<<endl;
    cout<<"        material name = "<<material_name[i]<<endl;
    cout<<"        quantity = "<<quantity[i]<<endl;
    cout<<"        material unit = "<<unit[i]<<endl;
    cout<<"        minimum stock range = "<<minimum_stock[i]<<endl;
    cout<<"        unit price = "<<unit_price[i]<<endl;
    cout<<"        supplier name = "<<supplier_name[i]<<endl;
    cout<<"        stock status = "<<stock_status[i]<<endl;

    cout<<"\n=====
    =\n\n";
}

}

public:
    void update_stock()
    {
        string materiall_id;
        string materials_name;
    again:
        if(product_number == 0)
        {
            cout<<"no materials available\n";

```

```

    return;
}
cout<<"enter material id\n";
cin>>materiall_id;
for(i=0; i<product_number; i++)
{
    if(materiall_id == material_id[i])
    {
        material:
        cout<<"enter material name\n";
        getline(cin>>ws, materials_name);
        if(materials_name == material_name[i])
        {
            cout<<"enter quantity\n";
            cin>>material_quantity;
            if(material_quantity < 0)
            {
                cout<<"quantity cannot be negative\n";
                goto material;
            }
            quantity[i] = quantity[i] + material_quantity;
            if(quantity[i] < minimum_stock[i])
            {
                stock_status[i] = "LOW STOCK";
            }
            else if(quantity[i] == minimum_stock[i])
            {
                stock_status[i] = "MINIMUM STOCK";
            }
        }
    }
}

```

```

    }

    else

    {

        stock_status[i] = "STOCK FULL";

    }

    cout<<"stock updated successfully\n";

    return;

}

else

{

    cout<<"wrong material name\n";

    goto material;

}

}

}

cout<<"material id not found\n";

goto again;

}

public:

void show_update_stock()

{

    string materiall_id;

    cout<<"=====
=====\\n\\n";

    cout<<"          REPORT\\n";

```

```

cout<<"\n=====
=====\\n\\n";

    if(product_number == 0)
    {
        cout<<"no materials available\\n";

cout<<"\n=====
=====\\n\\n";

        return;
    }

    cout<<"enter material id\\n";
    cin>>materiall_id;
    for(i=0; i<product_number; i++)
    {
        if(materiall_id == material_id[i])
        {
            cout<<"        MATERIAL ID = "<<material_id[i]<<endl;
            cout<<"        MATERIAL NAME = "<<material_name[i]<<endl;
            cout<<"        ENTERED QUANTITY = "<<material_quantity<<endl;
            cout<<"        TOTAL STOCK = "<<quantity[i]<<endl;
            cout<<"        STOCK STATUS = "<<stock_status[i]<<endl;
cout<<"\n=====
=====\\n\\n";

            return;
        }
    }

    cout<<"material not found\\n";

```

```

cout<<"\n=====
=====\\n\\n";

}

public:

void check_stock()

{

    string materialll_id;

    string materialss_name;

correction:

    if(product_number == 0)

    {

        cout<<"no materials available\\n";

        return;

    }

    cout<<"enter material id\\n";

    cin>>materialll_id;

    for(i=0; i<product_number; i++)

    {

        if(materialll_id == material_id[i])

        {

            corrections:

                cout<<"enter material name\\n";

                getline(cin>>ws, materialss_name);


                if(materialss_name == material_name[i])

                {

                    cout<<"material found\\n";

                    return;

```

```

    }

    else
    {
        cout<<"wrong material name\n";
        goto corrections;
    }
}

}

cout<<"material id not found\n";
goto correction;
}

public:

void show_check_stock()
{
    string materialll_id;
    cout<<"=====
=====\\n\\n";

    cout<<"          REPORT\\n";

    cout<<"\\n=====
=====\\n\\n";

    if(product_number == 0)
    {
        cout<<"no materials available\\n";

        cout<<"\\n=====
=====\\n\\n";

        return;
    }
}

```

```

        cout<<"enter material id\n";
        cin>>materialll_id;
        for(i=0; i<product_number; i++)
        {
            if(materialll_id == material_id[i])
            {
                cout<<"MATERIAL ID = "<<material_id[i]<<endl;
                cout<<"MATERIAL NAME = "<<material_name[i]<<endl;
                cout<<"TOTAL STOCK = "<<quantity[i]<<endl;
                cout<<"MINIMUM STOCK = "<<minimum_stock[i]<<endl;
                cout<<"STOCK STATUS = "<<stock_status[i]<<endl;
            }
            cout<<"\n=====
            =====\n\n";
            return;
        }
    }

    cout<<"material not found\n";

    cout<<"\n=====
    =====\n\n";

}

public:
    void showmaterial()
    {
        cout<<"=====MATERIALS LIST=====\\n";
        if(product_number == 0)
        {
            cout<<"no materials available\n";
            return;
        }
    }
}

```

```

    }

    for(i=0; i<product_number; i++)
    {
        cout<<i+1<<". MATERIAL ID = "<<material_id[i]<<endl;
        cout<<" MATERIAL NAME = "<<material_name[i]<<endl;
    }
}

};

```

// 6. SENSOR MONITORING CLASS

```

class sensor_monitoring : public machine_management

```

```

{

```

```

private:

```

```

    string sensor_id;
    string machine_id_check;
    string sensor_type;
    float sensor_value = 0;
    float minimum_value = 0;
    float maximum_value = 0;
    string sensor_unit;
    string sensor_status;

```

```

public:

```

```

    sensor_monitoring() : machine_management() {}
    sensor_monitoring(string factory_name)
        : machine_management(factory_name) {}

```

```

public:

```

```

    void add_sensor(string registered_machine_id)
    {
        cout<<"enter sensor id\n";
    }

```

```
cin>>sensor_id;
```

```
rewrite:
```

```
cout<<"enter machine id\n";
```

```
cin>>machine_id_check;
```

```
if(machine_id_check != registered_machine_id)
```

```
{
```

```
    cout<<"wrong machine id\n";
```

```
    cout<<"please enter the registered machine id\n";
```

```
    goto rewrite;
```

```
}
```

```
else
```

```
{
```

```
    cout<<"enter sensor type\n";
```

```
    cin>>sensor_type;
```

```
    cout<<"enter sensor value\n";
```

```
    cin>>sensor_value;
```

```
    cout<<"enter sensor unit\n";
```

```
    cin>>sensor_unit;
```

```
}
```

```
}
```

```
public:
```

```
void showsensor()
```

```
{
```

```
cout<<"=====
=====\\n\\n";
```

```
    cout<<"          SENSOR REPORT\\n";
```

```

cout<<"\n=====
=====\\n\\n";

cout<<"SENSOR ID = "<<sensor_id<<endl;
cout<<"MACHINE ID = "<<machine_id_check<<endl;
cout<<"SENSOR TYPE = "<<sensor_type<<endl;
cout<<"SENSOR VALUE = "<<sensor_value<<endl;
cout<<"SELECTED UNIT = "<<sensor_unit<<endl;
if(sensor_type == "temperature")
{
    if(sensor_value >= 40 CC sensor_value <= 70)
    {
        sensor_status = "NORMAL";
    }
    else if(sensor_value > 70 CC sensor_value <= 85)
    {
        sensor_status = "WARNING";
    }
    else if(sensor_value > 85)
    {
        sensor_status = "CRITICAL";
    }
    else
    {
        sensor_status = "INVALID";
    }
}
else if(sensor_type == "pressure")
{

```

```
if(sensor_value >= 2 CC sensor_value <= 6)
{
    sensor_status = "NORMAL";
}
else if(sensor_value > 6 CC sensor_value <= 8)
{
    sensor_status = "WARNING";
}
else if(sensor_value > 8)
{
    sensor_status = "CRITICAL";
}
else
{
    sensor_status = "INVALID";
}
}
else if(sensor_type == "speed")
{
    if(sensor_value >= 1000 CC sensor_value <= 2000)
    {
        sensor_status = "NORMAL";
    }

    else if(sensor_value > 2000 CC sensor_value <= 2200)
    {
        sensor_status = "WARNING";
    }
}
```

```
else if(sensor_value > 2200)
{
    sensor_status = "CRITICAL";
}

else
{
    sensor_status = "INVALID";
}
}
else if(sensor_type == "vibration")
{
    if(sensor_value >= 0 CC sensor_value <= 4)
    {
        sensor_status = "NORMAL";
    }
    else if(sensor_value > 4 CC sensor_value <= 7)
    {
        sensor_status = "WARNING";
    }
    else if(sensor_value > 7)
    {
        sensor_status = "CRITICAL";
    }
    else
    {
        sensor_status = "INVALID";
    }
}
```

```

    }
}
else
{
    sensor_status = "UNKNOWN SENSOR TYPE";
}
cout<<"SENSOR STATUS = "<<sensor_status<<endl;

cout<<"\n=====
=====\\n\\n";

}
};

// 7. QUALITY CONTROL CLASS
class quality_control : public factory
{
private:
    string quality_id;
    string production_id;
    string product_name;
    string product_type;
    int total_quantity = 0;
    int passed_quantity = 0;
    int failed_quantity = 0;
    float weight = 0;
    float length = 0;
    float width = 0;
    float height = 0;
    float pass_percentage = 0;
    float rejection_percentage = 0;

```

```

string defect_type;

string inspection_result;

string quality_status;

public:

    quality_control() : factory() {}

    quality_control(string factory_name) : factory(factory_name) {}

public:

    void quality_check()

    {

cout<<"=====
=====\\n";

        cout<<"===== QUALITY CONTROL MANAGEMENT SYSTEM
=====\\n";

        cout<<"enter quality id\\n";
        getline(cin>>ws, quality_id);
        cout<<"enter production id\\n";
        getline(cin>>ws, production_id);
        cout<<"enter product name\\n";
        getline(cin>>ws, product_name);
        cout<<"enter product type\\n";
        getline(cin>>ws, product_type);
        cout<<"enter total quantity\\n";
        cin>>total_quantity;
        cout<<"enter passed quantity\\n";
        cin>>passed_quantity;
        cout<<"enter failed quantity\\n";
        cin>>failed_quantity;

```

```

cout<<"enter product weight\n";
cin>>weight;
cout<<"enter product length\n";
cin>>length;
cout<<"enter product width\n";
cin>>width;
cout<<"enter product height\n";
cin>>height;
cout<<"enter defect type\n";
getline(cin>>ws, defect_type);
if(total_quantity > 0 CC
    passed_quantity >= 0 CC
    failed_quantity >= 0 CC
    passed_quantity + failed_quantity == total_quantity)
{
    pass_percentage =
        ((float)passed_quantity / total_quantity) * 100;
    rejection_percentage =
        ((float)failed_quantity / total_quantity) * 100;
    if(failed_quantity == 0)
    {
        quality_status = "PASSED";
        inspection_result = "ALL PRODUCTS PASSED QUALITY CHECK";
    }
    else if(failed_quantity < total_quantity)
    {
        quality_status = "WARNING";
        inspection_result = "SOME PRODUCTS FAILED QUALITY CHECK";
    }
}

```

```

    }
    else
    {
        quality_status = "FAILED";
        inspection_result = "ALL PRODUCTS FAILED QUALITY CHECK";
    }
}
else
{
    quality_status = "INVALID";
    inspection_result = "INVALID QUANTITY DATA";
    pass_percentage = 0;
    rejection_percentage = 0;
}
}

public:
    void show_quality_report()
    {

        cout<<"=====
        =====\n\n";

        cout<<"          QUALITY CONTROL REPORT\n";

        cout<<"=====
        =====\n\n";

        cout<<"QUALITY ID = "<<quality_id<<endl;
        cout<<"PRODUCTION ID = "<<production_id<<endl;
        cout<<"PRODUCT NAME = "<<product_name<<endl;
        cout<<"PRODUCT TYPE = "<<product_type<<endl;
    }
}

```

```

cout<<"TOTAL QUANTITY = "<<total_quantity<<endl;
cout<<"PASSED QUANTITY = "<<passed_quantity<<endl;
cout<<"FAILED QUANTITY = "<<failed_quantity<<endl;
cout<<"PASS PERCENTAGE = "<<pass_percentage<<"%"<<endl;
cout<<"REJECTION PERCENTAGE = "<<rejection_percentage<<"%"<<endl;
cout<<"PRODUCT WEIGHT = "<<weight<<endl;
cout<<"PRODUCT LENGTH = "<<length<<endl;
cout<<"PRODUCT WIDTH = "<<width<<endl;
cout<<"PRODUCT HEIGHT = "<<height<<endl;
cout<<"DEFECT TYPE = "<<defect_type<<endl;
cout<<"INSPECTION RESULT = "<<inspection_result<<endl;
cout<<"QUALITY STATUS = "<<quality_status<<endl;

cout<<"\n=====
=====\\n\\n";

}

};

// EMPLOYEE MANAGEMENT CLASS
class employee_management : public factory
{
private:
    factory* factory_ref;

public:
    employee_management() : factory(), factory_ref(nullptr) {} // nullptr = empty pointer
    employee_management(factory Cf) : factory(), factory_ref(Cf) {} // f=obj, C = reference

    void add_employee()
    {

```

```

if(factory_ref == nullptr)
{
    cout << "\nfactory not connected!";
    return;
}

int number;

cout << "\nenter number of employees you want to add: ";
cin >> number;

if(number <= 0)
{
    cout << "\ninvalid number of employees!";
    return;
}

for(int i = 0; i < number; i++)
{
    factory_ref->add_one_employee_from_management();
}

cout << "\n\n total employees added: "
    <<  factory_ref->get_factory_employee_count();

cout << "\n\nmaximum employees: "
    <<  factory_ref->get_total_employees();
}

```

```
void calculate_salary()
{
    if(factory_ref == nullptr)
    {
        cout << "\nfactory not connected!";
        return;
    }

    factory_ref->calculate_factory_salary();
}
```

```
void attendance()
{
    if(factory_ref == nullptr)
    {
        cout << "\nfactory not connected!";
        return;
    }
}
```

```
factory_ref->factory_attendance();
}
```

```
void show_employee()
{
    if(factory_ref == nullptr)
    {
        cout << "\nfactory not connected!";
    }
}
```

```

        return;
    }

    factory_ref->show_factory_employee();
}

void employee_report()
{
    if(factory_ref == nullptr)
    {
        cout << "\nfactory not connected!";
        return;
    }

    factory_ref->factory_employee_report(); // use of pointer
}

};

// 8. MAINTENANCE MANAGEMENT CLASS
class maintenance_management : public factory
{
private:
    string maintenance_id;
    string machine_id;
    string machine_name;
    string maintenance_type;
    // Preventive / Corrective / Emergency
    string problem_fault;
    string maintenance_date;

```

```

string technician_name;

float maintenance_cost = 0;

string maintenance_status;

// Pending / In Progress / Completed

string parts_replaced;

string next_maintenance_date;

public:

    maintenance_management() : factory() {}

    maintenance_management(string factory_name) : factory(factory_name) {}

public:

    void add_maintenance()

    {

cout<<"=====
=====\\n";

        cout<<"===== MAINTENANCE MANAGEMENT SYSTEM =====\\n";

cout<<"=====
=====\\n\\n";

        cout<<"enter maintenance id\\n";
        getline(cin>>ws, maintenance_id);
        cout<<"enter machine id\\n";
        getline(cin>>ws, machine_id);
        cout<<"enter machine name\\n";
        getline(cin>>ws, machine_name);

maintenance_type_input:

        cout<<"\\nenter maintenance type\\n";

        cout<<"1. Preventive\\n";

        cout<<"2. Corrective\\n";

```

```

cout<<"3. Emergency\n";
cout<<"enter your choice\n";
int type_choice;
cin>>type_choice;
if(type_choice == 1)
{
    maintenance_type = "Preventive";
}
else if(type_choice == 2)
{
    maintenance_type = "Corrective";
}
else if(type_choice == 3)
{
    maintenance_type = "Emergency";
}
else
{
    cout<<"invalid maintenance type\n";
    goto maintenance_type_input;
}
cout<<"\nenter problem / fault\n";
getline(cin>>ws, problem_fault);
cout<<"enter maintenance date\n";
getline(cin>>ws, maintenance_date);
cout<<"enter technician name\n";
getline(cin>>ws, technician_name);
cost:

```

```
cout<<"enter maintenance cost\n";
cin>>maintenance_cost;
if(maintenance_cost < 0)
{
    cout<<"maintenance cost cannot be negative\n";
    goto cost;
}
```

status_input:

```
cout<<"\nenter maintenance status\n";
cout<<"1. Pending\n";
cout<<"2. In Progress\n";
cout<<"3. Completed\n";
cout<<"enter your choice\n";
int status_choice;
cin>>status_choice;
if(status_choice == 1)
{
    maintenance_status = "Pending";
}
else if(status_choice == 2)
{
    maintenance_status = "In Progress";
}
else if(status_choice == 3)
{
    maintenance_status = "Completed";
}
else
```

```

{
    cout<<"invalid maintenance status\n";
    goto status_input;
}

cout<<"\nenter parts replaced\n";
getline(cin>>ws, parts_replaced);
cout<<"enter next maintenance date\n";
getline(cin>>ws, next_maintenance_date);
cout<<"\nmaintenance record added successfully\n";

cout<<"\n=====
=====\\n\\n";

}

public:

void show_maintenance()

{

cout<<"=====
=====\\n";

    cout<<"          MAINTENANCE REPORT\\n";

cout<<"=====
=====\\n\\n";

    cout<<"MAINTENANCE ID = "<<maintenance_id<<endl;
    cout<<"MACHINE ID = "<<machine_id<<endl;
    cout<<"MACHINE NAME = "<<machine_name<<endl;
    cout<<"MAINTENANCE TYPE = "<<maintenance_type<<endl;
    cout<<"PROBLEM / FAULT = "<<problem_fault<<endl;
    cout<<"MAINTENANCE DATE = "<<maintenance_date<<endl;
    cout<<"TECHNICIAN NAME = "<<technician_name<<endl;

```

```

    cout<<"MAINTENANCE COST = "<<maintenance_cost<<endl;
    cout<<"MAINTENANCE STATUS = "<<maintenance_status<<endl;
    cout<<"PARTS REPLACED = "<<parts_replaced<<endl;
    cout<<"NEXT MAINTENANCE DATE = "<<next_maintenance_date<<endl;

    cout<<"\n=====
=====\\n\\n";

}

public:

    void update_maintenance()
    {
        string id;

        cout<<"=====
=====\\n";

        cout<<"          UPDATE MAINTENANCE\\n";

        cout<<"=====
=====\\n\\n";

        if(maintenance_id == "")
        {
            cout<<"no maintenance record available\\n";
            return;
        }

        cout<<"enter maintenance id\\n";
        cin>>id;
        if(id != maintenance_id)
        {
            cout<<"maintenance id not found\\n";
            return;
        }
    }

```

```
}
```

```
update_status:
```

```
cout<<"\ncurrent maintenance status = "
```

```
<<maintenance_status<<endl;
```

```
cout<<"\nselect new maintenance status\n";
```

```
cout<<"1. Pending\n";
```

```
cout<<"2. In Progress\n";
```

```
cout<<"3. Completed\n";
```

```
cout<<"enter your choice\n";
```

```
int choice;
```

```
cin>>choice;
```

```
if(choice == 1)
```

```
{
```

```
    maintenance_status = "Pending";
```

```
}
```

```
else if(choice == 2)
```

```
{
```

```
    maintenance_status = "In Progress";
```

```
}
```

```
else if(choice == 3)
```

```
{
```

```
    maintenance_status = "Completed";
```

```
}
```

```
else
```

```
{
```

```
    cout<<"invalid choice\n";
```

```
    goto update_status;
```

```
}
```

```

        cout<<"\nmaintenance status updated successfully\n";

        cout<<"new maintenance status = "
            <<maintenance_status<<endl;

    cout<<"\n=====
=====\\n\\n";

    }

public:

    void maintenance_report()
    {
        cout<<"=====
=====\\n";

        cout<<"          MAINTENANCE MANAGEMENT REPORT\\n";

        cout<<"=====
=====\\n\\n";

        if(maintenance_id == "")
        {
            cout<<"no maintenance record available\\n";

            cout<<"\n=====
=====\\n\\n";

            return;
        }

        cout<<"MAINTENANCE ID   = "<<maintenance_id<<endl;
        cout<<"MACHINE ID      = "<<machine_id<<endl;
        cout<<"MACHINE NAME      = "<<machine_name<<endl;
        cout<<"MAINTENANCE TYPE   = "<<maintenance_type<<endl;
        cout<<"PROBLEM / FAULT    = "<<problem_fault<<endl;
        cout<<"MAINTENANCE DATE    = "<<maintenance_date<<endl;
        cout<<"TECHNICIAN NAME     = "<<technician_name<<endl;
    }

```

```

cout<<"MAINTENANCE COST  = "<<maintenance_cost<<endl;
cout<<"MAINTENANCE STATUS = "<<maintenance_status<<endl;
cout<<"PARTS REPLACED    = "<<parts_replaced<<endl;
cout<<"NEXT MAINTENANCE  = "<<next_maintenance_date<<endl;

cout<<"\n-----\n";

if(maintenance_status == "Pending")
{
    cout<<"MAINTENANCE ACTION = MAINTENANCE IS PENDING\n";
}
else if(maintenance_status == "In Progress")
{
    cout<<"MAINTENANCE ACTION = MAINTENANCE WORK IS CURRENTLY IN
PROGRESS\n";
}
else if(maintenance_status == "Completed")
{
    cout<<"MAINTENANCE ACTION = MAINTENANCE HAS BEEN COMPLETED\n";
}

if(maintenance_type == "Emergency")
{
    cout<<"PRIORITY = HIGH\n";
    cout<<"ACTION = IMMEDIATE MAINTENANCE REQUIRED\n";
}
else if(maintenance_type == "Corrective")
{
    cout<<"PRIORITY = MEDIUM\n";
    cout<<"ACTION = FAULT CORRECTION REQUIRED\n";
}

```

```

        else if(maintenance_type == "Preventive")
        {
            cout<<"PRIORITY = ROUTINE\n";
            cout<<"ACTION = SCHEDULED PREVENTIVE MAINTENANCE\n";
        }
        cout<<"\n=====
=====\\n\\n";
    }

public:
    string get_maintenance_id()
    {
        return maintenance_id;
    }
    string get_machine_id()
    {
        return machine_id;
    }
    string get_maintenance_status()
    {
        return maintenance_status;
    }
};

// 9. REPORT CLASS
class report
{
public:
    void factory_report(factory Cf)
    {
        cout << "\\n===== FACTORY REPORT =====\\n";
    }

```

```

        f.show_factory();
    }
    void machine_report(machine_management Cm)
    {
        cout << "\n===== MACHINE REPORT =====\n";
        m.showmachine();
    }
    void production_report(production_management Cp)
    {
        cout << "\n===== PRODUCTION REPORT =====\n";
        p.showproduction();
    }
    void inventory_report(inventory Ci)
    {
        cout << "\n===== INVENTORY REPORT =====\n";
        i.showentry();
    }
    void employee_report(employee_management Ce)
    {
        cout << "\n===== EMPLOYEE REPORT =====\n";
        e.employee_report();
    }
    void quality_report(quality_control Cq)
    {
        cout << "\n===== QUALITY REPORT =====\n";
        q.show_quality_report();
    }
    void maintenance_report(maintenance_management Cmm)

```

```

{
    cout << "\n===== MAINTENANCE REPORT =====\n";
    mm.maintenance_report();
}

void complete_report(
    factory Cf,
    machine_management Cm,
    production_management Cp,
    inventory Ci,
    employee_management Ce,
    quality_control Cq,
    maintenance_management Cmm)
{
    cout << "\n";
    cout <<
    "===== \n";
    cout << "        COMPLETE FACTORY REPORT\n";
    cout <<
    "===== \n";

    cout << "\n\n***** FACTORY INFORMATION *****\n";
    f.show_factory();
    cout << "\n\n***** MACHINE INFORMATION *****\n";
    m.showmachine();
    cout << "\n\n***** PRODUCTION INFORMATION *****\n";
    p.showproduction();
    cout << "\n\n***** INVENTORY INFORMATION *****\n";
    i.showentry();
    cout << "\n\n***** EMPLOYEE INFORMATION *****\n";
    e.employee_report();
}

```

```

        cout << "\n\n***** QUALITY INFORMATION *****\n";
        q.show_quality_report();
        cout << "\n\n***** MAINTENANCE INFORMATION *****\n";
        mm.maintenance_report();

        cout <<
        "\n===== \n";

        cout << "          END OF COMPLETE FACTORY REPORT\n";

        cout <<
        "===== \n";

    }
};

```

// 10. SALES REPORT CLASS

```

class sales_report : public inventory
{
private:
    string sale_machine_id;
    string sale_machine_name;
    int sale_machine_qty;
    float sale_unit_price;
    float total_sales_report;
    float gst_amount;
    float base_amount;
    float final_amount;
public:
    sales_report() : inventory()
    {
        sale_machine_qty = 0;
    }
};

```

```

    sale_unit_price = 0;
    total_sales_report = 0;
    gst_amount = 0;
    base_amount = 0;
    final_amount = 0;
}

public:

void generate_sales_report(inventory Cinv)
{

cout<<"=====
=====\\n";

    cout<<"          SALES REPORT\\n";

cout<<"=====
=====\\n\\n";


    cout<<"enter material id\\n";
    getline(cin>>ws, sale_machine_id);
    cout<<"enter material name\\n";
    getline(cin>>ws, sale_machine_name);
    cout<<"enter quantity sold\\n";
    cin>>sale_machine_qty;
    // CHECK PRODUCT IN INVENTORY
    if(inv.sell_material(sale_machine_id, sale_machine_name,
        sale_machine_qty, sale_unit_price) == false)
    {
        cout << "\\nsales cannot be generated because product is not available.\\n";
        return;
    }
}

```

```

    }

    // BASE AMOUNT
    base_amount = sale_machine_qty * sale_unit_price;

    // GST 2%
    gst_amount = (base_amount * 2) / 100;

    // FINAL AMOUNT
    final_amount = base_amount + gst_amount;

    total_sales_report = final_amount;

    cout<<"\nsales report generated successfully\n";

    cout<<"\n=====
=====\\n";

    }

public:

    void show_sales_report()

    {

    cout<<"=====
=====\\n\\n";

        cout<<"          SALES REPORT\\n";

    cout<<"=====
=====\\n\\n";

        if(sale_machine_id == "")

        {

            cout<<"no sales report available\\n";

    cout<<"\n=====
=====\\n\\n";

            return;

        }

    cout<<"SALE MACHINE ID   = "<<sale_machine_id<<endl;

    cout<<"SALE MACHINE NAME   = "<<sale_machine_name<<endl;

```

```

        cout<<"QUANTITY SOLD    = "<<sale_machine_qty<<endl;
        cout<<"UNIT PRICE      = "<<sale_unit_price<<endl;
        cout<<"BASE AMOUNT     = "<<base_amount<<endl;
        cout<<"GST (2%)       = "<<gst_amount<<endl;
        cout<<"TOTAL SALES     = "<<final_amount<<endl;

        cout<<"\n=====
        =====\n\n";

    }

public:
    float get_total_sales()
    {
        return total_sales_report;
    }
};

// MAIN MENU
void menu()
{
    factory f;
    production_management n;
    login l;
    inventory inv;
    sensor_monitoring s;
    quality_control qc;
    employee_management e(f);
    maintenance_management mm;
    report r;
    sales_report sr;
    int choice;

```

```

int continue_code;

bool password_created = false;

bool logged_in = false;

int inventory_choice;

// FACTORY REGISTRATION FIRST

cout<<"=====
=====\\n\\n";

cout<<"    SMART MANUFACTURING AND PRODUCTION AUTOMATION\\n";

cout<<"\\n=====
=====\\n\\n";

f.register_factory();

f.show_factory();

f.add_factory_employee();

// CREATE PASSWORD

l.create();

password_created = true;

logged_in = l.login_id(f.get_phone_no());

if(logged_in == false)
{
    cout<<"login failed\\n";

    return;
}

// CREATE OBJECTS FOR REGISTERED FACTORY

inv = inventory(f.factory_name); // stores the entered values

machine_management m(f.factory_name);

s = sensor_monitoring(f.factory_name);

```

```

qc = quality_control(f.factory_name);
e.factory_name = f.factory_name;
e.set_total_employees(f.get_total_employees());
mm.factory_name = f.factory_name;
restart:

cout<<"=====
=====\\n\\n";

    cout<<"    SMART MANUFACTURING AND PRODUCTION AUTOMATION\\n";
cout<<"\\n=====
=====\\n\\n";

    cout<<"        1. factory registration\\n";
    cout<<"        2. machine management\\n";
    cout<<"        3. production management\\n";
    cout<<"        4. inventory management\\n";
    cout<<"        5. sensor monitoring\\n";
    cout<<"        6. quality control\\n";
    cout<<"        7. employee management\\n";
    cout<<"        8. maintenance management\\n";
    cout<<"        9. reports\\n";
    cout<<"       10. change password\\n";
    cout<<"       11. generate sales report\\n";
    cout<<"       12. exit\\n";
    cout<<"enter your choice\\n";
    cin>>choice;
    switch(choice)
    {
        // FACTORY REGISTRATION
        case 1:

```

```

{
    f.register_factory();
    f.add_factory_employee();
    e.factory_name = f.factory_name;
e.set_total_employees(f.get_total_employees());
f.show_factory();
f.show_factory_employee();
    inv.factory_name = f.factory_name;
    m.factory_name = f.factory_name;
    s.factory_name = f.factory_name;
    n.factory_name = f.factory_name;
    qc.factory_name = f.factory_name;
    mm.factory_name = f.factory_name;
    cout<<"factory registration updated successfully\n";
    cout<<"do you want to continue\n";
    cout<<" press 1 for continue\n";
    cout<<" press 2 for stop\n";
    cin>>continue_code;
    if(continue_code == 1)
    {
        goto restart;
    }
    else
    {
        cout<<"=====THANK YOU FOR VISITING=====\\n";
    }
    break;
}

```

```
// MACHINE MANAGEMENT
```

```
case 2:
```

```
{  
    if(password_created == false || logged_in == false)  
    {  
        cout<<"please complete factory registration and login first\n";  
        goto restart;  
    }  
    m.machine();  
    m.showmachine();  
    cout<<"do you want to continue\n";  
    cout<<" press 1 for continue\n";  
    cout<<" press 2 for stop\n";  
    cin>>continue_code;  
    if(continue_code == 1)  
    {  
        goto restart;  
    }  
    else  
    {  
        cout<<"=====THANK YOU FOR VISITING=====\\n";  
    }  
}
```

```
break;
```

```
}
```

```
// PRODUCTION MANAGEMENT
```

```
case 3:
```

```
{
```

```

if(password_created == false || logged_in == false)
{
    cout<<"please complete factory registration and login first\n";
    goto restart;
}
n.production();
n.showproduction();
cout<<"do you want to continue\n";
cout<<" press 1 for continue\n";
cout<<" press 2 for stop\n";
cin>>continue_code;
if(continue_code == 1)
{
    goto restart;
}
else
{
    cout<<"=====THANK YOU FOR VISITING=====\\n";
}
break;
}

// INVENTORY MANAGEMENT
case 4:
{
    if(password_created == false || logged_in == false)
    {
        cout<<"please complete factory registration and login first\n";
        goto restart;
    }
}

```

```
}
```

inventory_menu:

```
cout<<"=====
=====\\n";
```

```
    cout<<"        INVENTORY MANAGEMENT\\n";
```

```
cout<<"=====
=====\\n\\n";
```

```
    cout<<"        1. add stock\\n";
```

```
    cout<<"        2. show all materials\\n";
```

```
    cout<<"        3. update stock\\n";
```

```
    cout<<"        4. check stock\\n";
```

```
    cout<<"        5. show material list\\n";
```

```
    cout<<"        6. back to main menu\\n";
```

```
    cout<<"enter your choice\\n";
```

```
    cin>>inventory_choice;
```

```
    switch(inventory_choice)
```

```
{
```

```
    case 1:
```

```
{
```

```
    inv.add_stock();
```

```
    goto inventory_menu;
```

```
}
```

```
    case 2:
```

```
{
```

```
        inv.showentry();  
        goto inventory_menu;  
    }  
    case 3:  
    {  
        inv.update_stock();  
        inv.show_update_stock();  
        goto inventory_menu;  
    }  
    case 4:  
    {  
        inv.check_stock();  
        inv.show_check_stock();  
        goto inventory_menu;  
    }  
    case 5:  
    {  
        inv.showmaterial();  
        goto inventory_menu;  
    }  
    case 6:  
    {  
        goto restart;  
    }  
    default:  
    {  
        cout<<"invalid input\n";  
        goto inventory_menu;  
    }
```

```

    }
}
break;
}
// SENSOR MONITORING
case 5:
{
    if(password_created == false || logged_in == false)
    {
        cout<<"please complete factory registration and login first\n";
        goto restart;
    }
    if(m.get_machine_id() == "")
    {
        cout<<"please register machine first\n";
        goto restart;
    }
    s.add_sensor(m.get_machine_id());
    s.showsensor();
    cout<<"do you want to continue\n";
    cout<<" press 1 for continue\n";
    cout<<" press 2 for stop\n";
    cin>>continue_code;
    if(continue_code == 1)
    {
        goto restart;
    }
    else

```

```

{
    cout<<"=====THANK YOU FOR VISITING=====\\n";
}

break;
}

// QUALITY CONTROL
case 6:
{
    if(password_created == false || logged_in == false)
    {
        cout<<"please complete factory registration and login first\\n";
        goto restart;
    }

    qc.quality_check();
    qc.show_quality_report();
    cout<<"do you want to continue\\n";
    cout<<" press 1 for continue\\n";
    cout<<" press 2 for stop\\n";
    cin>>continue_code;
    if(continue_code == 1)
    {
        goto restart;
    }
    else
    {
        cout<<"=====THANK YOU FOR VISITING=====\\n";
    }

    break;
}

```

```

    }

    // EMPLOYEE MANAGEMENT

    case 7:
    {
        if(password_created == false || logged_in == false)
        {
            cout<<"please complete factory registration and login first\n";
            goto restart;
        }
    }

    employee_menu:

    cout << "\n===== \n";
    cout << "    employee management\n";
    cout << "===== \n";

    cout << "\n1. add employee";
    cout << "\n2. calculate salary";
    cout << "\n3. attendance";
    cout << "\n4. show employee";
    cout << "\n5. employee report";
    cout << "\n6. back to main menu";
    cout << "\n\nenter your choice: ";

    cin >> choice;

    switch(choice)
    {
        case 1:
        {
            e.add_employee();
            goto employee_menu;
        }
    }

```

case 2:

```
{  
    e.calculate_salary();  
    goto employee_menu;  
}
```

case 3:

```
{  
    e.attendance();  
    goto employee_menu;  
}
```

case 4:

```
{  
    e.show_employee();  
    goto employee_menu;  
}
```

case 5:

```
{  
    e.employee_report();  
    goto employee_menu;  
}
```

case 6:

```
{  
    goto restart;  
}
```

default:

```
{  
    cout << "\ninvalid choice!";
```

```

        goto employee_menu;
    }
}

// MAINTENANCE MANAGEMENT
// MAINTENANCE MANAGEMENT
case 8:
{
    if(password_created == false || logged_in == false)
    {
        cout<<"please complete factory registration and login first\n";
        goto restart;
    }

maintenance_menu:

cout<<"=====
=====\\n";

    cout<<"          MAINTENANCE MANAGEMENT\\n";

cout<<"=====
=====\\n\\n";

    cout<<"          1. add maintenance\\n";
    cout<<"          2. show maintenance\\n";
    cout<<"          3. update maintenance\\n";
    cout<<"          4. maintenance report\\n";
    cout<<"          5. back to main menu\\n";

    cout<<"enter your choice\\n";

    cin>>choice;

    switch(choice)
    {

        // ADD MAINTENANCE

```

```
case 1:
{
    mm.add_maintenance();

    goto maintenance_menu;
}
// SHOW MAINTENANCE
case 2:
{
    mm.show_maintenance();

    goto maintenance_menu;
}
// UPDATE MAINTENANCE
case 3:
{
    mm.update_maintenance();
    goto maintenance_menu;
}
// MAINTENANCE REPORT
case 4:
{
    mm.maintenance_report();
    goto maintenance_menu;
}
// BACK TO MAIN MENU
case 5:
{
```

```

        goto restart;
    }
    default:
    {
        cout<<"invalid input\n";
        goto maintenance_menu;
    }
}
break;
}

// REPORTS
case 9:
{
    if(password_created == false || logged_in == false)
    {
        cout<<"please complete factory registration and login first\n";
        goto restart;
    }
}
report_menu:
cout<<"\n===== \n";
;
    cout<<"          REPORTS\n";

cout<<"===== \n\n";
;
    cout<<"          1. Factory Report\n";
    cout<<"          2. Machine Report\n";
    cout<<"          3. Production Report\n";
    cout<<"          4. Inventory Report\n";

```

```

cout<<"          5. Employee Report\n";
cout<<"          6. Quality Report\n";
cout<<"          7. Maintenance Report\n";
cout<<"          8. Complete Factory Report\n";
cout<<"          9. Back to Main Menu\n";
cout<<"\nenter your choice\n";
cin>>choice;
switch(choice)
{
    // FACTORY REPORT
    case 1:
    {
        r.factory_report(f);
        goto report_menu;
    }
    // MACHINE REPORT
    case 2:
    {
        r.machine_report(m);
        goto report_menu;
    }
    // PRODUCTION REPORT
    case 3:
    {
        r.production_report(n);
        goto report_menu;
    }
    // INVENTORY REPORT

```

```
case 4:
{
    r.inventory_report(inv);
    goto report_menu;
}
// EMPLOYEE REPORT
case 5:
{
    r.employee_report(e);
    goto report_menu;
}
// QUALITY REPORT
case 6:
{
    r.quality_report(qc);
    goto report_menu;
}
// MAINTENANCE REPORT
case 7:
{
    r.maintenance_report(mm);
    goto report_menu;
}
// COMPLETE FACTORY REPORT
case 8:
{
    r.complete_report(f, m, n, inv, e, qc, mm);
    goto report_menu;
```

```

    }

    // BACK TO MAIN MENU

    case 9:

    {

        goto restart;

    }

    default:

    {

        cout<<"invalid input\n";

        goto report_menu;

    }

}

break;

}

// CHANGE PASSWORD

case 10:

{

    if(password_created == false || logged_in == false)

    {

        cout<<"please login first\n";

        goto restart;

    }

    l.change_password(f.get_phone_no());

    cout<<"do you want to continue\n";

    cout<<" press 1 for continue\n";

    cout<<" press 2 for stop\n";

    cin>>continue_code;

    if(continue_code == 1)

```

```

    {
        goto restart;
    }
    else
    {
        cout<<"=====THANK YOU FOR VISITING=====\\n";
    }
    break;
}

// GENERATE SALES REPORT
case 11:
{
    if(password_created == false || logged_in == false)
    {
        cout<<"please complete factory registration and login first\\n";
        goto restart;
    }
    sr.factory_name = f.factory_name;
    sr.generate_sales_report(inv);
    sr.show_sales_report();
    cout<<"do you want to continue\\n";
    cout<<" press 1 for continue\\n";
    cout<<" press 2 for stop\\n";
    cin>>continue_code;

    if(continue_code == 1)
    {
        goto restart;
    }
}

```

```

    }
    else
    {
        cout<<"=====THANK YOU FOR VISITING=====\\n";
    }
    break;
}

// EXIT
case 12:
{
    cout<<"=====THANK YOU FOR VISITING=====\\n";
    break;
}
default:
{
    cout<<"invalid input\\n";
    goto restart;
}
}

}

// MAIN FUNCTION
int main()
{
    menu();
    return 0;
}

```